Lab 5
AI
Topic: Explain and utilize Python packages such as NumPy, Pandas, Scikit-learn, and Matplotlib.

Objective

The goal of this lab is to help students understand how to work with **NumPy, Pandas, Scikit-learn, and Matplotlib**. including their creation, manipulation, and mathematical operations.

1. Introduction to Arrays in Python

What is an Array?

An array is a collection of similar data types stored in contiguous memory locations. Unlike lists, arrays are optimized for numerical computations, making them faster and more efficient for large datasets.

Introduction to NumPy

NumPy (Numerical Python) is a powerful library for numerical computing in Python. It provides support for **multi-dimensional arrays** and **mathematical functions** that are optimized for performance.

Installing NumPy

Before using NumPy, you need to install it using pip:

pip install numpy

Importing NumPy

To use NumPy in Python, you need to import it:

import numpy as np

2. Creating NumPy Arrays

1- Creating a NumPy Array from a List

NumPy (Numerical Python) is a powerful library in Python used for numerical computations.One of its most fundamental features is the **numpy.array()** function, which allows us to create arrays from lists. NumPy arrays are more efficient than Python lists, providing faster processing and additional functionalities.

Why Use NumPy Arrays Instead of Python Lists?

1. **Performance:** NumPy arrays are stored in contiguous memory locations, making operations faster compared to Python lists.
2. **Convenience:** NumPy provides built-in mathematical functions that work directly on arrays without needing loops.
3. **Memory Efficiency:** NumPy arrays consume less memory than Python lists.

Syntax

```
import numpy as np
array_name = np.array([list_elements])
```

- **import numpy as np** → Imports the NumPy library.
- **np.array([list_elements])** → Converts a Python list into a NumPy array

Code:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5]) # Creating a 1D array from a Python list
print(arr)
```

2- Creating an Array Using arange()

What is arange()?

arange() is a NumPy function used to create an array with a sequence of numbers.

Why Use arange()?

It creates arrays quickly.
It allows step values, including decimal steps.
It is more efficient than using lists.

➢ Syntax

numpy.arange(start, stop, step, dtype)

- **start** → (Optional) The number to start from (default = 0).
- **stop** → The number to stop at (exclusive).
- **step** → (Optional) The gap between numbers (default = 1).
- **dtype** → (Optional) Defines the type of numbers.

Code:

```
import numpy as np
arr = np.arange(1, 10, 2)  # Creates an array with values from 1 to 9 with a step of 2
print(arr)  # Output: [1 3 5 7 9]
```

➢ **Specifying Data Type (dtype)**

```
import numpy as np
arr = np.arange(1, 10, 2, dtype=float)
print(arr)
```

3- Creating an Array Using linspace()

What is linspace()?

linspace() is a NumPy function that creates an array with evenly spaced values between a start and stop value. Unlike **arange()**, it specifies the number of values instead of the step size.

Why Use linspace()?

It ensures exactly **n** numbers between a given range.
It is useful in scientific calculations and graph plotting.

Syntax
numpy.linspace(start, stop, num, endpoint, dtype)

- **start** → The starting number of the sequence.
- **stop** → The ending number of the sequence.
- **num** → (Optional) The number of values.
- **endpoint** → (Optional) If True (default), includes the stop value.
- **dtype** → (Optional) Specifies the data type.

Code:

```
import numpy as np
```

```
arr = np.linspace(0, 10, 5)  # Generates 5 evenly spaced numbers between 0 and 10
print(arr)  # Output: [ 0.   2.5  5. 7.5 10. ]
```

4- Creating an Array Using zeros() and ones()

zeros(): Creates an array filled with **0s**.
ones(): Creates an array filled with **1s**.

These functions are useful for initializing arrays before performing calculations.

Code:

```
import numpy as np
arr_zeros = np.zeros((3,3))  # Creates a 3x3 array filled with zeros
arr_ones = np.ones((2,2))    # Creates a 2x2 array filled with ones
print(arr_zeros)
print(arr_ones)
```

- **Can We Use Other Numbers Like 2, 3, 4?**

No, **zeros()** and **ones()** only create arrays with **0s** and **1s**.
For other numbers, you can use **full()**:

Example:

```
import numpy as np
arr = np.full((3,3), 5)  # Creates a 3x3 array filled with 5s
print(arr)
```

5- Creating a Random Array

```
arr_rand = np.random.rand(3, 3)  # Creates a 3x3 array with random values between 0 and 1
print(arr_rand)
```

3. Understanding NumPy Array Properties

NumPy arrays have useful properties that help in data manipulation:

1. ndim **(Number of Dimensions)** – Tells how many dimensions the array has.
2. shape **(Dimensions of Array)** – Gives the number of rows and columns.

3. size **(Total Elements)** – Shows the total number of elements in the array.
4. dtype **(Data Type)** – Displays the data type of array elements.

Code:

```
arr = np.array([[1, 2, 3], [4, 5, 6]])  # Creating a 2D array

print("Data Type:", arr.dtype)   # Checks the data type of elements
print("Shape:", arr.shape)       # Returns (rows, columns)
print("Size:", arr.size)         # Total number of elements in array
print("Dimensions:", arr.ndim)   # Number of dimensions (1D, 2D, etc.)
```

4. Basic NumPy Operations

Arithmetic Operations

```
import numpy as np
arr = np.array([1, 2, 3])
print("Addition:", arr + 5)   # Adds 5 to each element
print("Multiplication:", arr * 2)  # Multiplies each element by 2
```

Broadcasting

Broadcasting is a **powerful feature** in NumPy that allows operations between arrays of **different shapes** without needing to manually resize them. It automatically expands smaller arrays to match the shape of larger ones
Or
we can say that it add corresponding elements in array

Code:

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
print("Element-wise Addition:", arr1 + arr2)  # Adds corresponding elements
```

Array Slicing and Indexing

1. Indexing (Accessing Elements)

Indexing in Python means **accessing elements** from a sequence (like lists, tuples, strings, or NumPy arrays) using their position (index)

NumPy uses **zero-based indexing** (first element is at index 0).

Example:

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])
print(arr[0])  # Output: 10 (First element)
print(arr[-1]) # Output: 50 (Last element)
```

2. Slicing (Extracting a Subset)

Slicing helps get multiple elements from an array.

Syntax: array[start:stop:step]

- start → Starting index (inclusive)
- stop → Ending index (exclusive)
- step → Steps to skip elements

Example:

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print(arr[1:4])  # Extracts elements from index 1 to 3 (20, 30, 40)
```

5. Advanced NumPy Operations

Mathematical Functions:

NumPy provides built-in mathematical functions to perform operations on arrays efficiently.

Example:

```
import numpy as np
arr = np.array([1, 4, 9, 16])
print("Square Root:", np.sqrt(arr))  # Calculates square root of each element
print("Logarithm:", np.log(arr))  # Computes natural log of each element
```

Statistical Operations

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print("Mean:", np.mean(arr))  # Computes the mean
print("Median:", np.median(arr))  # Computes the median
```

```
print("Standard Deviation:", np.std(arr))  # Computes the standard deviation
```

6. Reshaping and Manipulating NumPy Arrays

Reshaping Arrays:

Reshaping means changing the shape (dimensions) of a NumPy array without changing its data.

Example:

```
arr = np.array([1, 2, 3, 4, 5, 6])
reshaped_arr = arr.reshape(2, 3)  # Reshapes into a 2x3 matrix
print(reshaped_arr)
```

How it Works?

1. arr.reshape(2, 3) converts the **1D array** into a **2D array** with **2 rows and 3 columns**.
2. The total number of elements **must match** (6 elements remain 6).
3. NumPy automatically arranges elements row-wise.

Concatenation :

Concatenation means joining two or more NumPy arrays into one.

Example:

```
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
print("Concatenated Array:", np.concatenate((arr1, arr2)))  # Merges arrays
```

➢ Introduction to Pandas

Pandas is a Python library used for data analysis and manipulation. It provides data structures such as Series and DataFrames, which allow efficient handling of tabular data.

Why Use Pandas?

Pandas is widely used because:

1. It simplifies data handling and manipulation.
2. It works seamlessly with other Python libraries like NumPy and Matplotlib.
3. It supports data import and export from multiple formats such as CSV, Excel, SQL, and JSON.
4. It provides powerful data filtering, grouping, and visualization capabilities.

Installing Pandas in Google Colab

Google Colab already has Pandas pre-installed. However, if you are working in another environment, you can install it using:

pip install pandas

What is a DataFrame?

A DataFrame is a 2D labeled data structure similar to a table or spreadsheet. It consists of rows and columns, where each column can contain different data types such as integers, floats, and strings.
Think of a DataFrame as an Excel sheet where each row represents an entry, and each column represents an attribute.

Creating DataFrames

Method 1: Creating a DataFrame from a List

A DataFrame can be created from a list of lists, where each sublist represents a row of data.

Example:

```
import pandas as pd
# Creating a DataFrame from a list
data = [['Ali', 22], ['Ayesha', 21], ['Usman', 23]]
df = pd.DataFrame(data, columns=['Name', 'Age'])
print(df)
```

Explanation:

1. first import the pandas library.
2. We define a list of lists where each sublist contains Name and Age.
3. We use pd.DataFrame() to create a DataFrame with column names 'Name' and 'Age'.
4. Finally, we print the DataFrame.

Method 2: Creating a DataFrame from a Dictionary

A dictionary can be used to create a DataFrame, where keys become column names and values are lists of column data.

Example:

```
import pandas as pd
# Creating DataFrame from a dictionary
data = {
   'Name': ['Ali', 'Ayesha', 'Usman'],
   'Age': [22, 21, 23],
   'Grade': ['A', 'B', 'A']
}
df = pd.DataFrame(data)
print(df)
```

Explanation:

1. A dictionary is created with column names as keys and lists as values.
2. The DataFrame is created using pd.DataFrame(data).
3. The structure automatically aligns the data into tabular form.

Method 3: Creating a DataFrame from a CSV File

A DataFrame can also be created by reading data from a CSV(Comma Separated values)file.

Example:

```
import pandas as pd

# Load DataFrame from CSV
df = pd.read_csv('students.csv')
print(df.head())
```

Explanation:

1. The pd.read_csv() function reads a CSV file and converts it into a DataFrame.
2. The .head() function displays the first five rows of the DataFrame.

➢ Alternative Methods to Display Data

1. Show the First N Rows

You can specify how many rows you want:

```
print(df.head(10))
  # Shows the first 10 rows
```

2. Show the Last Rows

Use **tail()** to display the last rows:

```
print(df.tail())  # Shows the last 5 rows
print(df.tail(10))  # Shows the last 10 rows
```

3. Show a Random Sample

To display **random rows** from the dataset:

```
print(df.sample(5))  # Shows 5 random rows
```

➢ Accessing Data in a DataFrame

To retrieve specific columns or rows from a DataFrame, we use indexing.
Example:

```
print(df['Name'])  # Get a single column
print(df[['Name', 'Age']])  # Get multiple columns
```

Explanation:
1. df['Name'] retrieves a single column.
2. df[['Name', 'Age']] retrieves multiple columns as a DataFrame.

➢ Filtering and Sorting Data

Pandas allows filtering rows based on conditions and sorting values in ascending or descending order.
Example:

```
print(df[df['Age'] > 21])  # Show students older than 21
print(df.sort_values(by='Age'))  # Sort by Age (ascending)
df.sort_values(by='Age', ascending=False) #sort by decending
```

Explanation:
1. df[df['Age'] > 21] filters rows where Age is greater than 21.
2. df.sort_values(by='Age') sorts the DataFrame by Age in ascending order.

➢ Grouping and Aggregation

Pandas allows grouping data and applying aggregate functions like mean, sum, or count.
Example:

```
print(df.groupby('Grade').size())  # Count students per grade
print(df.groupby('Grade')['Age'].mean())  # Average age per grade
```

Explanation:

1. df.groupby('Grade').size() counts the number of students in each grade.
2. df.groupby('Grade')['Age'].mean() calculates the average age per grade.